Angular Component Creation Flow

Component + Lifecycle Hooks + Parent-Child Flow

August 20, 2026

TOPIC — WHAT "ANGULAR CREATES A COMPONENT" ACTUALLY MEANS

Angular Core | Component Lifecycle

What "Angular creates a component" actually means

Break it into 4 distinct mechanical steps - these are NOT lifecycle hooks, they're the actual engine work. Lifecycle hooks are just callbacks Angular invokes at specific checkpoints inside this process - they don't do the creation, they let you run code when a checkpoint is reached.

```
1. Instantiate - new YourComponentClass() runs (constructor executes,
   DI resolves dependencies via inject())
2. Set inputs - values from parent binding assigned to @Input()/input() properties
3. Create view - template compiled to DOM instructions get executed,
   actual DOM nodes created for this component's template
4. Change detection - bindings evaluated, DOM updated with real values,
   {{ }} interpolations filled in, [prop]="value" bindings applied
```

Lifecycle hooks are Angular saying: "I just finished step X, here's a callback if you want to react to it." That's the entire relationship - hooks are notifications about engine milestones, not the milestones themselves.

Single component - hook-by-hook mapping to the 4 steps

Engine step	Hook fired	What's actually true at this point
1. Instantiate	(constructor itself)	Class exists, but no inputs, no DOM, no ch
2. Set inputs	ngOnChanges (if @Input() used)	Inputs available, DOM still doesn't exist
-	ngOnInit	Inputs settled, safe to use them, DOM still
-	ngDoCheck	Custom change-detection point, still pre-D
3. Create view	(DOM nodes created here - no hook exists for this exact instant)	-
-	ngAfterContentInit	Content projected via <ng-content> is now
-	ngAfterContentChecked	-
-	ngAfterViewInit	This component's entire own template DO
-	ngAfterViewChecked	-

Key insight: there's a real gap - DOM creation itself has no dedicated hook. ngOnInit fires before DOM exists. ngAfterViewInit fires after DOM exists. Nothing fires exactly during. That gap is why afterNextRender() exists - it's the closest thing to "DOM is not just created but painted."

Parent with in its template - full merged flow

Your parent's template contains <app-child>. That tag is a placeholder Angular resolves during parent's own "create view" step (step 3). This is the critical link - child creation is triggered from inside parent's view-creation step, not after it.

```
PARENT step 1: constructor runs
PARENT step 2: inputs set -> ngOnChanges -> ngOnInit -> ngDoCheck
PARENT step 3: create view - Angular walks parent's template DOM instructions
    -> hits <app-child> -> this is where CHILD creation begins, nested inside
      parent's own step 3:

    CHILD step 1: constructor runs
    CHILD step 2: inputs set (from parent's [prop]="..." bindings)
                  -> ngOnChanges -> ngOnInit -> ngDoCheck
    CHILD step 3: create view - child's own template DOM built and
                  inserted inside <app-child> host element
    CHILD step 4: change detection runs on child's bindings
    CHILD hooks: ngAfterContentInit -> ngAfterContentChecked
                 -> ngAfterViewInit -> ngAfterViewChecked
                 (child is now FULLY done, top to bottom)

    <- back to parent's step 3, <app-child> placeholder now fully resolved
PARENT step 3 continues: rest of parent's template DOM built
PARENT step 4: change detection on parent's own bindings
PARENT hooks: ngAfterContentInit -> ngAfterContentChecked
              -> ngAfterViewInit -> ngAfterViewChecked
```

The one sentence that ties it together

Parent's step 3 (create view) is not one atomic action - it's a template walk, and every child selector tag encountered during that walk triggers the entire 4-step process, recursively, for that child, before parent's walk can continue past that point. That's why child's ngOnInit always fires before parent's ngAfterViewInit, but always after parent's own ngOnInit - parent's early hooks (OnInit, DoCheck) happen before the template walk even starts, while parent's late hooks (AfterViewInit) can only fire after the walk - including every recursive child call inside it - fully completes.

■■

Additional Context Worth Knowing

Where DOM creation happens (Ivy mechanics)

Step 3 ("create view") is powered by Ivy's compiled instruction set - each component template compiles down to functions like elementStart(), text(), elementEnd() that run once at creation time to build the DOM skeleton, separate from the update instructions (textInterpolate, property) that CD later calls repeatedly. Creation mode and update mode are two distinct instruction sets Ivy generates per template - this is why Angular can cheaply re-run only the update instructions on every CD pass without rebuilding DOM nodes from scratch.

Static vs dynamic ViewChild timing exception

@ViewChild(Foo, { static: true }) resolves in ngOnInit, not ngAfterViewInit - because static: true tells Angular the queried element isn't behind a structural directive (@if, @for), so it can be created eagerly during the parent's own creation step rather than waiting for content projection/child resolution. Signal-based viewChild() has no static option - it is always resolved lazily and safely returns undefined until ready, since reading a signal never throws.

Why this matters for afterNextRender()

Because no hook fires exactly at "DOM created and painted," any DOM-measurement or third-party DOM integration logic placed in ngAfterViewInit runs before the browser has actually painted the frame - it only guarantees DOM nodes exist in memory, not that layout/paint has occurred. afterNextRender() is scheduled after the browser's real paint, making it the safe hook for getBoundingClientRect(), chart libraries, or focus management.

Destruction is the mirror image

Component teardown runs bottom-up too: ngOnDestroy fires on children before their parent, for the same structural reason creation runs top-down - a parent cannot safely destroy itself while children still hold references into its injector or DOM subtree.